

교과목 2 : 데이터 분석과 머신러닝 / 딥러닝

단원 2 : 머신러닝

DAY2. 머신러닝 회귀모델과 분류모델

목 차

1. PyTorch 프레임워크		3
2. 회귀 및 분류모델		9
3. 지도학습 머신러닝 분류모델		13
4. 지도학습 머신러닝 회귀모델		24

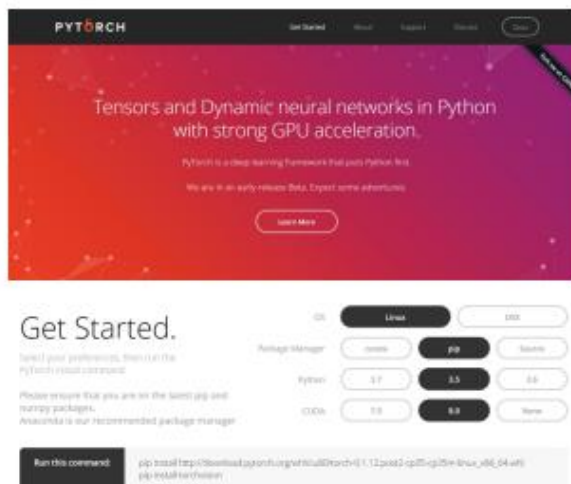
1. PyTorch 프레임워크



Yann LeCun
DIRECTOR OF AI RESEARCH
Facebook AI Research (FAIR)



Soumith Chintala
RESEARCH ENGINEER
Facebook AI Research (FAIR)



<파이토치의 특징>

- Python first
- GPU acceleration
- Linux/osx



NumPy

VS

PYTORCH

```

In [4]: import numpy as np
from datetime import datetime
start = datetime.now()

np.random.seed(0)

N, D = 3, 4

x = np.random.randn(N, D)
y = np.random.randn(N, D)
z = np.random.randn(N, D)

a = x + y
b = a + z
c = np.sum(b)

grad_c = 1.0
grad_b = grad_c * np.ones((N, D))
grad_a = grad_b.copy()
grad_x = grad_a + y
grad_y = grad_a + x

print(grad_x)
print(grad_y)
print(grad_z)
print(datetime.now() - start)

```

Gradient
Calculation

```

In [6]: import torch
from torch.autograd import Variable
from datetime import datetime
start = datetime.now()

N, D = 3, 4

x = Variable(torch.randn(N, D).cuda(), requires_grad=True)
y = Variable(torch.randn(N, D).cuda(), requires_grad=True)
z = Variable(torch.randn(N, D).cuda(), requires_grad=True)

a = x + y
b = a + z
c = torch.sum(b)

c.backward(torch.ones_like(c).cuda())

print(grad_x)
print(grad_y)
print(grad_z)
print(datetime.now() - start)

Variable containing:
-0.6089  0.6640 -1.3035  1.0094
-0.0701 -0.0702 -0.0404 -1.7546
-0.3297  0.6298  2.5136 -0.5987
[torch.cuda.FloatTensor of size 3x4 (GPU 0)]

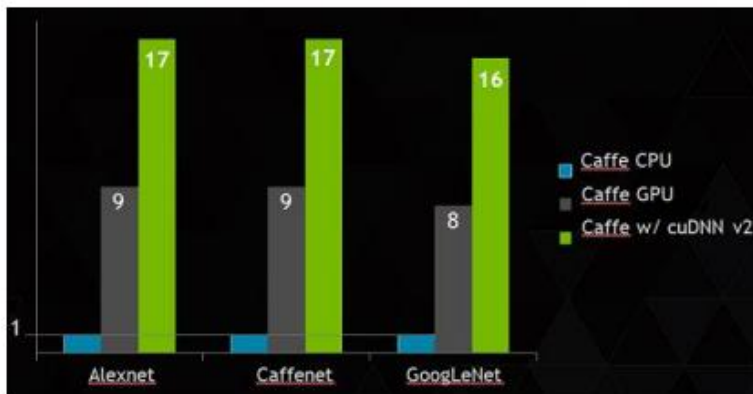
Variable containing:
0.1152  1.1808  1.4522  1.9419
1.0645  0.1587 -1.6528  0.4038
1.5935 -0.4028 -1.4324 -0.7998
[torch.cuda.FloatTensor of size 3x4 (GPU 0)]

Variable containing:
1  1  1
1  1  1
1  1  1
[torch.cuda.FloatTensor of size 3x4 (GPU 0)]

0:00:00.003434

```

걸리는 시간은 비슷(?)



(출처: Nvidia)

<Numpy vs PyTorch>

- 연산량이 많을수록 성능 차이가 커짐
- Automatic Gradient calculation!!
- 모델 자체에 집중할 수 있음

```
[6]: import torch
from torch.autograd import Variable
from datetime import datetime
start = datetime.now()

N, D = 3, 4

x = Variable(torch.randn(1, D).cuda()), requires_grad=True
y = Variable(torch.randn(1, D).cuda()), requires_grad=True
z = Variable(torch.randn(1, D).cuda()), requires_grad=True

a = x * y
b = a * 2
c = torch.sum(b)

c.backward() # grad is torch.cuda.FloatTensor([1, 0, 0])

print(x.grad)
print(y.grad)
print(z.grad)
print(datetime.now()-start)

Variable containing:
-0.8008  0.8640 -1.8005  1.0994
-0.0731 -0.0702 -0.0474 -1.7546
-0.3247  0.6230  2.5105 -0.5907
[torch.cuda.FloatTensor at size 3x4 (GPU 0)]

Variable containing:
0.1152  1.3804  1.4522  1.9417
1.0846  0.1587 -1.6525  0.4031
1.9905 -0.4729 -1.4024 -0.7008
[torch.cuda.FloatTensor at size 3x4 (GPU 0)]

Variable containing:
1  1  1  1
1  1  1  1
1  1  1  1
[torch.cuda.FloatTensor at size 3x4 (GPU 0)]
```

TF가 10배 이상 느리다??

- Define by Run
- Dynamic graph
- 늘어나는 추세
- 자체적 포럼 운영
- PyTorch-KR

Packages

Package	Description
torch	a Tensor library like NumPy, with strong GPU support
torch.autograd	a tape based automatic differentiation library that supports all differentiable Tensor operations in torch
torch.nn	a neural networks library deeply integrated with autograd designed for maximum flexibility
torch.optim	an optimization package to be used with torch.nn with standard optimization methods such as SGD, RMSProp, LBFGS, Adam etc.
torch.multiprocessing	python multiprocessing, but with magical memory sharing of torch Tensors across processes. Useful for data loading and hogwild training.
torch.utils	DataLoader, Trainer and other utility functions for convenience
torch.legacy(.nn/.optim)	legacy code that has been ported over from torch for backward compatibility reasons



torchvision

torchvision Reference

- torchvision
- torchvision.datasets
- torchvision.models
- torchvision.transforms
- torchvision.utils

많이 사용되는 데이터들

- ImageFolder(data loader)
- MNIST
- COCO
- LSUN
- Imagenet-12
- CIFAR10 and CIFAR100
- STL10

torchvision Reference

- torchvision
- torchvision.datasets
- torchvision.models
- torchvision.transforms
- torchvision.utils

많이 사용되는 데이터들

많이 사용되는 모델들

- AlexNet
- VGG
- ResNet
- SqueezeNet
- DenseNet

Packages

torchvision Reference

- torchvision
- torchvision.datasets
- torchvision.models
- torchvision.transforms
- torchvision.utils

많이 사용되는 데이터들

많이 사용되는 모델들

많이 사용되는 이미지 변환들

- Scale
- Pad
- Crop
- Normalize
- Flip

Packages

torchvision Reference

- torchvision
- torchvision.datasets
- torchvision.models
- torchvision.transforms
- torchvision.utils

많이 사용되는 데이터들

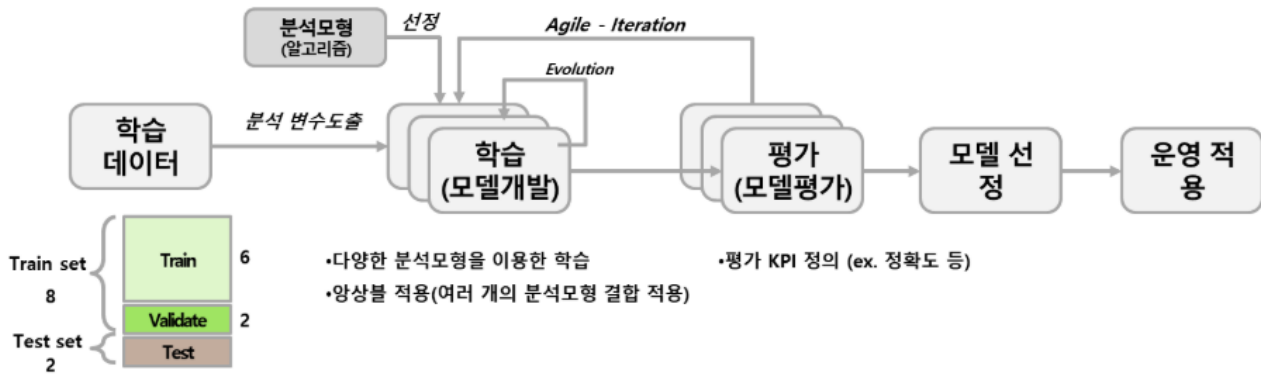
많이 사용되는 모델들

많이 사용되는 이미지 변환들

(많이 사용되는) 이미지 저장 함수

2. 회귀 및 분류모델

참조.모델링 순서



이미지의 전체 흐름은 다음과 같다.

학습 데이터 준비

↓

분석모형(알고리즘) 선택

↓

학습(모델 개발)

↓

평가(모델 평가)

↓

모델 선정

↓

운영 적용

즉,

데이터 준비 → AI 학습 → 성능 평가 → 좋은 모델 선택 → 실제 서비스 사용
과정이다.

분석 알고리즘 유형

1. 분석 알고리즘



2. 예측 유형

- 선형회귀분석 (Linear Regression)
- 의사결정나무 (Decision Tree)
- 인공신경망 (Artificial Neural Network)
- SVR (Support Vector Regression)
- KNN (K-Nearest Neighborhood) 등

분석 알고리즘 유형

3. 분류 유형

- 로지스틱회귀분석 (Logistic Regression)
- 의사결정나무 (Decision Tree)
- 인공신경망 (Artificial Neural Network)
- SVC (Support Vector Classifier)
- KNN (K-Nearest Neighborhood) 등

회귀모델 개요

1. 회귀모델

- 수치형 종속 값을 예측하는 지도학습 모델
- 선형회귀모델이 대표적임

2. 선형 회귀모델 실습

- ① 데이터 분리
- ② Linear Regression으로 선형회귀모델 학습
- ③ 예측 수행
- ④ 모델 평가
- ⑤ 결과 시각화

분류모델 개요

1. 분류모델

- 명목형 종속 값을 분류하는 지도학습 모델
- 로지스틱회귀모델이 대표적임

2. 로지스틱회귀모델

- 독립변수의 선형결합을 이용해 사건의 발생 여부를 예측함
- 종속변수가 범주형일 때 사용하는 회귀분석 모델

3. 분류모델 실습

- ① 아이리스 데이터셋 불러오기
- ② 독립변수와 종속변수 설정
- ③ 로지스틱회귀모델 학습
- ④ 예측 수행
- ⑤ 모델 평가
- ⑥ 시각화 - 차트 그리기

3. 지도학습 머신러닝 분류모델

데이터 탐색

1. 타이타닉 데이터 분류모델링 실습

- 데이터 탐색 시각화 전처리 영역
- 머신러닝 모델링과 평가의 영역

2. 데이터파일 올리기

- 타이타닉 데이터 다운로드 받기
- 콜랩 왼쪽 메뉴바에서 폴더 아이콘 선택하기
- 상단 아이콘 파일 업로드하기

3. 데이터 로드

- 데이터 로드 → 데이터 확인

4. 데이터 탐색

- ① 데이터 구조 확인
- ② 결측치 확인
- ③ 결측치 처리
- ④ 기술 통계 확인
- ⑤ 변수 정리

5. 데이터 시각화

- ① 여성 vs 남성 생존자 수 비교
- ② 생존율 히스토그램
- ③ Pclass 별 생존율 비교
- ④ Embarked 별 생존율 비교
- ⑤ Age 히스토그램
- ⑥ Fare 히스토그램
- ⑦ 상관관계 히트맵 (코릴레이션 매트릭스)

데이터 전처리

1. 데이터 전처리 개념

- 데이터를 탐색적으로 분석 시 결측치 처리
- 결측치를 미리 처리한 이유 : 원하는 시각화의 형태를 얻기 위함

2. 데이터 전처리 요소

- 결측치 처리
- 이상치 처리
- 인코딩(문자 값을 숫자 값으로 바꾸는 작업)
- 데이터 분리
- 파생 변수 생성

3. 데이터 전처리 실습

- ① 범주형 변수 레이블 인코딩
- ② 수치형 변수 스케일링
- ③ 학습 데이터와 테스트 데이터 분리

MISSION: 회귀 및 분류모델 분석해보기

데이터를 분석할 때 데이터 탐색을 먼저 해야 할까요,
데이터 전처리를 먼저해야 할까요?

데이터 분석

데이터 탐색

데이터 전처리

- ▶ 순서에 맞춰서 진행되기 보다는 병행 경향
- ▶ 예) 결측치 여부 확인: 데이터 먼저 탐색
데이터의 시각화 작업: 결측치 먼저 제거

분류모델링

1. 분류모델링 및 평가 실습

- ① Logistic Regression 모델
- ② Decision Tree 모델

2. 분류모델링 및 평가 완료 후

- 하이퍼 파라미터 튜닝 등의 모델 성능을 더 좋게 하는 방법이 있음

분류모델 지표

1. 혼동행렬(Confusion matrix)

- 분류모델의 성능 평가를 위해 사용되는 표
- 예측값과 실제값 간의 차이를 2차원 매트릭스로 나타냄
- 주로 이진 분류 문제에서 사용
- 다중 분류 문제에서도 확장하여 사용 가능

2. 평가지표

- 정확도(Accuracy)
- 정밀도(Precision)
- 재현율(Recall, Sensitivity)
- F1-score
- 특이도(Specificity)
- 거짓긍정률(FPR)
- 참긍정률(TPR)

3. ROC 커브

- 임계값(Threshold)을 0에서 1까지 변화시켜 가면서 x축에는 거짓 긍정률을, y축에는 참 긍정률을 표시한 곡선
- 아래 면적은 AUC(Area Under the Curve)라고 하며, AUC값이 1에 가까울수록 분류 모델의 성능이 좋다고 평가

MISSION: 분류모델의 평가지표

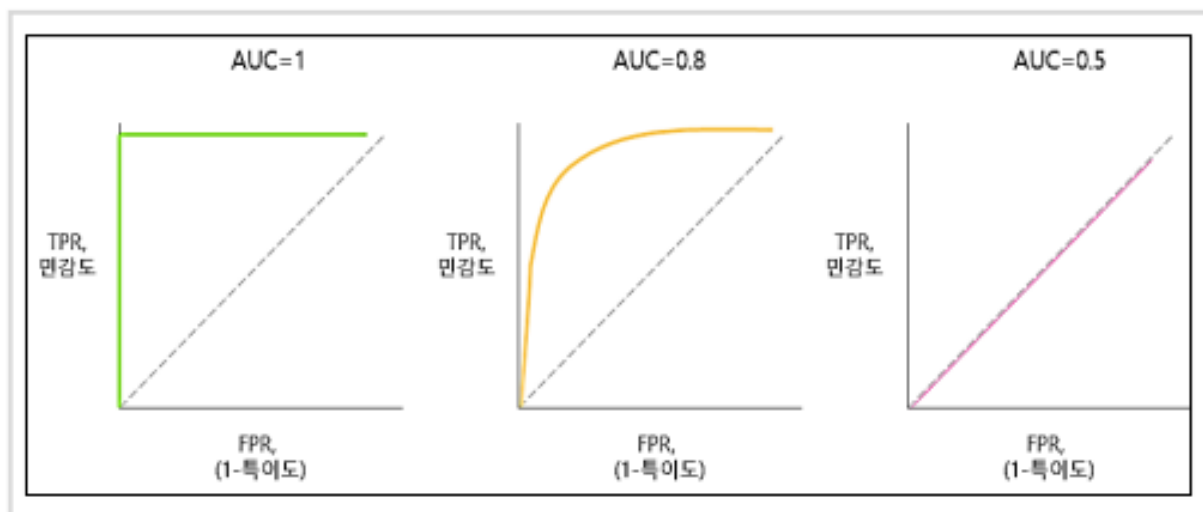
평가지표: ROC 커브 이해하기

- ROC 커브는 어떻게 만들어지는 걸까요?

ROC 커브

임계값(Threshold)을 0에서 1까지 변화시켜 가면서
x축에는 거짓 긍정률을, y축에는 참 긍정률을 표시한 곡선

- ▶ 분류모델은 0 아니면 1로 분류
- ▶ 확률의 기준을 0.5(50%)로 둬
- ▶ 50% 미만은 0, 50% 이상은 1로 분류
- ▶ 커브의 아래쪽 면적으로 모델의 성능을 평가



평가지표: ROC 커브

파라미터 vs 하이퍼파라미터

1. 파라미터

- 모델이 데이터로부터 찾아내는 값
- 최적의 모델을 만드는 게 중요
→ 모델이 학습 과정에서 찾아내는 설정 값들
- 설정 값은 사람이 설정하는 게 아님
- 학습 과정에서 자연스럽게 찾아 나가는 값

2. 하이퍼파라미터

- 모델 학습 과정 전에 사람이 직접 설정해 주는 값
→ 분석가가 직접 설정

3. 파라미터와 하이퍼파라미터 비교

구분	파라미터 (Parameter)	하이퍼파라미터 (Hyperparameter)
정의	모델이 데이터로부터 학습하는 값	모델 학습 과정에서 사람이 직접 설정해야 하는 값
변경 여부	데이터에 따라 자동으로 변경됨	모델 학습 전에 미리 설정해야 하며, 학습 과정에서 변경되지 않음
예시	선형 회귀의 가중치 (Weights), 절편 (Bias)	학습률 (Learning Rate), 트리 개수 (n_estimators), 정규화 강도 (alpha)
조정 방법	모델 학습을 통해 자동으로 조정됨	사용자가 설정한 값을 기반으로 모델이 학습함. 그리드 서치나 랜덤 서치 등을 통해 최적의 값을 찾을 수 있음
조정 시점	모델이 학습할 때 자동으로 최적화됨	모델이 학습을 시작하기 전에 사용자에게 의해 설정됨
조정 주체	모델이 데이터를 바탕으로 자동 조정	사용자가 설정하거나 튜닝 알고리즘을 통해 최적화 가능
영향	모델의 예측 성능에 직접적으로 영향을 미침	모델의 학습 과정이나 과적합/과소적합을 방지하는 데 중요한 역할을 함

하이퍼파라미터 튜닝 방식

1. Grid Search

- 가능한 모든 하이퍼파라미터의 조합을 시도하여 최적의 조합을 찾는 방법
- 많은 시간이 소요되고 많은 자원 사용되는 것이 단점

2. Random Search

- Grid Search의 단점을 보완해서 만들어짐
- 가장 훌륭한 하이퍼파라미터의 조합을 놓칠 수 있다는 것이 단점

3. 하이퍼파라미터 튜닝 방식 비교

비교 기준	그리드 서치 (Grid Search)	랜덤 서치 (Random Search)
설명	가능한 모든 하이퍼파라미터 조합을 시도하여 최적의 조합을 찾는 방법	하이퍼파라미터 공간에서 무작위로 샘플링하여 일부 조합을 시도하여 최적의 조합을 찾는 방법
장점	- 모든 조합을 시도하므로 최적의 하이퍼파라미터를 찾을 가능성이 높음	- 계산 비용 절감 가능 (전체 조합 중 일부만 시도)
단점	- 계산 비용이 매우 큼 (고차원 매개변수에서는 비효율적)	- 모든 조합을 시도하지 않으므로 최적의 하이퍼파라미터를 놓칠 가능성이 있음
적합성	- 파라미터 공간이 작을 때 적합	- 파라미터 공간이 매우 클 때 적합
탐색 방식	- 모든 가능한 조합을 체계적으로 탐색	- 무작위로 선택한 일부 조합을 탐색
계산 비용	- 매우 높음	- 상대적으로 낮음
시간 소요	- 모든 조합을 시도하기 때문에 매우 오래 걸림	- 그리드 서치에 비해 더 짧은 시간 소요
적용 상황	- 탐색할 파라미터의 범위가 좁고, 자원이 충분할 때 유리	- 탐색할 파라미터의 범위가 넓거나, 자원이 제한적일 때 유리
최적화 가능성	- 최적의 하이퍼파라미터를 찾을 가능성이 높음	- 최적의 하이퍼파라미터를 놓칠 수 있지만, 무작위로 탐색하면서 좋은 조합을 찾을 확률이 있음
초기 설정 필요	- 세밀한 하이퍼파라미터 설정이 필요	- 대략적인 범위만 설정하면 됨

하이퍼파라미터 튜닝 코드 비교

1. Grid Search 튜닝 코드

```
# 랜덤 포레스트 모델 정의
model = RandomForestRegressor(random_state=42)

# 그리드 서치에 사용할 하이퍼파라미터 범위 설정
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# 그리드 서치 수행
grid_search = GridSearchCV(estimator=model, param_grid=param_grid, cv=5, scoring='neg_mean')
grid_search.fit(X_train, y_train)

# 최적의 하이퍼파라미터와 성능 출력
print(f"Best parameters: {grid_search.best_params_}")
print(f"Best score: {grid_search.best_score_}")
```

하이퍼파라미터 튜닝 코드 비교

2. Random Search 튜닝 코드

```
# 랜덤 포레스트 모델 정의
model = RandomForestRegressor(random_state=42)

# 랜덤 서치에 사용할 하이퍼파라미터 범위 설정
param_dist = {
    'n_estimators': randint(100, 500),
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': randint(2, 11),
    'min_samples_leaf': randint(1, 5)
}

# 랜덤 서치 수행
random_search = RandomizedSearchCV(estimator=model, param_distributions=param_dist, n_iter=100)
random_search.fit(X_train, y_train)

# 최적의 하이퍼파라미터와 성능 출력
print(f"Best parameters: {random_search.best_params_}")
print(f"Best score: {random_search.best_score_}")
```

하이퍼파라미터 튜닝(그리드서치)

1. 하이퍼파라미터 튜닝 도구

- CPU로 이용 시 하이퍼파라미터 튜닝 시간이 오래 걸림
- GPU 이용을 위해 하드웨어 유형 변경 필요

2. 하이퍼파라미터 튜닝 실습

- ① 하이퍼파라미터 튜닝 범위 설정
- ② GridSearchCV 객체 생성
- ③ 그리드서치 수행
- ④ 최적의 하이퍼파라미터 출력
- ⑤ 최적의 모델 평가

3. 테스트 데이터 성능 측정

- ① 테스트 데이터 전처리
- ② 랜덤 포레스트 일반 모델 예측
- ③ 최적화된 랜덤 포레스트 모델 예측

MISSION: 지도학습 머신러닝 모델 분석해보기

하이퍼파라미터 튜닝을 해서 베스트 모델을 찾았음에도
베스트 모델의 성능이 더 떨어지는 경우도 있을까요?



4. 지도학습 머신러닝 회귀모델

데이터 탐색 개요 및 실습

1. 회귀 모델링

- 숫자값을 예측하기 위한 모델

2. 데이터 탐색 실습

- ① 데이터 파일 올리기
- ② 데이터 확인
- ③ 데이터 구조 확인
- ④ 결측치 확인
- ⑤ 결측치 처리
- ⑥ 결측치 처리 (visibility 평균값으로 대체)
- ⑦ 결측치 처리 (station_name unknown값으로 대체)
- ⑧ 기술통계확인
- ⑨ 변수 정리

3. 데이터 시각화 실습

- ① 데이터 파일 올리기
- ② 데이터 확인
- ③ 데이터 구조 확인
- ④ 결측치 확인
- ⑤ 결측치 처리
- ⑥ 결측치 처리 (visibility 평균값으로 대체)
- ⑦ 결측치 처리 (station_name unknown값으로 대체)
- ⑧ 기술통계확인
- ⑨ 변수 정리

데이터 시각화 개요 및 실습

1. 데이터 시각화

- 데이터를 잘 이해하기 위한 것
- 데이터의 건수가 많고 복잡하기 때문에 사용
- matplotlib와 seaborn 이용
- 판다스 등 다른 패키지 사용 가능

2. 데이터 시각화 실습

- ① 수치형 변수들에 대한 히스토그램 시각화
- ② 요일별 이용객 수 변화 시각화 (선 그래프)
- ③ 월별 이용객 수 변화 시각화 (선 그래프)
- ④ 수치형 변수들 간의 상관관계 시각화
- ⑤ 특정 수치형 변수와 목표 변수(num_people) 간의 관계 시각화 (예: 산점도)
- ⑥ 카테고리형 변수들의 분포 시각화 (예: 막대 그래프)
- ⑦ 카테고리형 변수와 목표 변수(num_people) 간의 관계 시각화 (박스 플롯)
- ⑧ 카테고리형 변수와 목표 변수(num_people) 간의 관계 시각화 (바이올린 플롯)

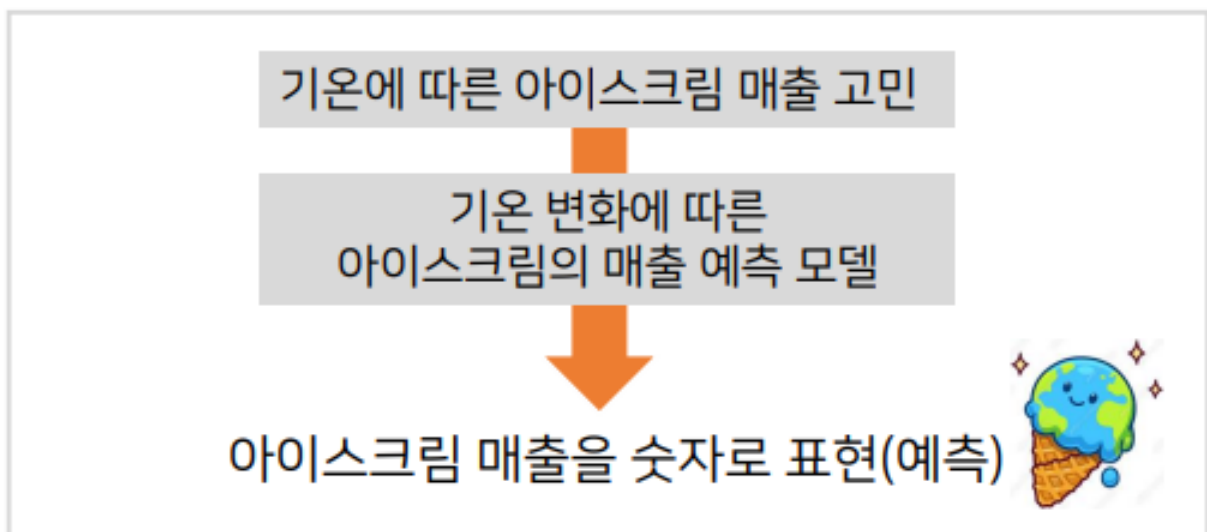
데이터 전처리 실습

1. 데이터 전처리 과정 실습

- ① 이상값 처리
- ② 라벨 인코딩
- ③ 수치형 변수 스케일링
- ④ 학습용 데이터셋 분할

회귀모델은 언제 어떻게 사용해야 할까요?

▶ 회귀모델은 숫자를 예측하는 모델



모델링 및 평가 튜닝하기

1. 데이터 시각화를 통한 데이터 이해

- ① 데이터 로드
- ② 데이터 탐색
- ③ 데이터 시각화
- ④ 데이터 전처리
- ⑤ 라벨 인코딩
- ⑥ 스케일링 작업

2. 모델링 및 평가 과정

- ① 학습용, 검증용 데이터 분리
- ② 모델링(선형 회귀모델, 리니어리그레션, 랜덤 포레스트, XGBRegressor)
- ③ 하이퍼파라미터 튜닝
- ④ 랜덤 서치 CV 이용
- ⑤ 베스트 모델 평가

회귀모델 지표

1. 회귀모델 평가 지표

- 회귀모델의 예측 결과는 수치로 나타남
- 실제 값과 예측 값과의 차이를 다양한 방식으로 평균하여 계산한 지표를 사용함
- 분석모형이 선형 회귀모델인 경우 결정계수를 추가 지표로 사용함

2. 회귀모델 평가 지표 종류

- 평균절대오차(MAE)
- 평균제곱오차(MSE)
- 평균제곱근오차(RMSE)
- 평균절대백분율오차(MAPE)
- 결정계수(R²)
- adjusted R²

[분석주제 및 데이터설명]

서울시에서 지하철역의 운영 효율성을 높이기 위해 각 역의 이용객 수를 예측하는 모델을 구축하려고 한다. 주어진 데이터를 바탕으로 지하철역의 이용객 수(num_people)를 예측하는 머신러닝 모델을 만들어, 이를 바탕으로 각 지하철역의 자원 배분 및 운영 계획을 수립할 예정이다.

변수명	설명	변수 유형
date	날짜	수치형
day_of_week	요일	범주형
month	월	수치형
station_name	지하철 역 이름	범주형
visibility	가시거리 (단위: km)	수치형
precipitation	강수량 (단위: mm)	수치형
temperature	온도 (단위: °C)	수치형
num_people	해당 지하철역의 이용객 수	수치형

회귀모델 지표 중 R2가 1에 가까울수록 더 좋은 모델일까요?

$R^2 =$ 결정계수

0과 1 사이에 존재

모델이 데이터를 얼마나 잘 설명하는지를 나타냄

1에 가까울수록 모델이 데이터를 더 잘 설명,
그렇다고 더 좋은 모델이라는 말은 아님

훈련용 데이터에 너무 적합된 상태 = 과적합 상태

과적합 상태에서는
테스트용 데이터를 잘 설명하지 못함